Aspera Orchestrator Workflow Authoring Guide

IBM Aspera

2026/10/21

Contents

1 Introduction **3**
1.1 Terminology . . . 3

2 Quick start **4**

3 Lifecycle **5**

4 Workflow file **6**
4.1 File types . . . 6
4.2 Structure . . . 6
4.3 Plain format . . . 7
4.4 Rails format . . . 7
4.5 Workflow attributes . . . 7
4.6 Whiteboard attributes . . . 8
4.7 Action templates . . . 8

5 Graph XML **9**
5.1 Skeleton . . . 9
5.2 Terminal nodes . . . 9
5.3 Step . . . 10
5.4 Prerequisites and branches . . . 11
5.5 Synchronization rules (AND vs OR) . . . 11
5.6 Inputs . . . 12
5.7 Outputs . . . 12
5.8 Parameters . . . 13
5.9 Value types . . . 13
5.10 Notes . . . 14

6 Inputs of a step **15**
6.1 Placeholders . . . 15
6.2 Plugin specifications . . . 16

7 Execution **18**
7.1 Statuses . . . 18
7.2 Retries . . . 18
7.3 Processing code . . . 18
7.4 Triggers . . . 19

8 Main action plugins **20**
8.1 CustomRuby . . . 20
8.2 Filter . . . 20
8.3 LocalFileWatcher . . . 21
8.4 ArrayFanout and Funnelin . . . 21

9 Recipes **22**
9.1 Recipe: minimal workflow . . . 22
9.2 Recipe: runtime parameter and step chaining . . . 22
9.3 Recipe: branch on a status . . . 23
9.4 Recipe: wait for a file . . . 24

9.5 Update an existing workflow . . . 25

10 Authoring checklist **26**

11 REST API **27**
11.1 import_workflow . . . 27
11.2 export_workflow . . . 27
11.3 publish_workflow . . . 28
11.4 find_constraints . . . 28
11.5 import_with_constraints . . . 28
11.6 initiate . . . 28
11.7 Other workflow endpoints . . . 29
11.8 ascli commands . . . 29

12 Known issues **30**

Chapter 1

Introduction

This guide explains how to write an Aspera Orchestrator workflow as a file, import it, publish it and run it, without the graphical designer and without exporting an existing workflow first.

It is written for people and for AI assistants. Each rule gives the exact element, attribute or value expected by Orchestrator.

The content is derived from the Orchestrator 4.1.6 source code. The examples in Recipes were imported, published and executed on Orchestrator 4.1.6.

Related documents:

- Action Template Reference: attributes, inputs and outputs of every action plugin (generated from the plugin source).
- Plugin specifications: the same as data (YAML), with the allowed values of attributes and the template settings each input and output depends on. Its header describes its fields. Read one plugin with `yq`: `yq '.AmazonS3Operation' plugins.yml`.
- Plugin Manual: functional description of each action plugin.
- Plugin Development Guide: how to write an action plugin.

1.1 Terminology

Term	Meaning
Workflow	A named process definition, identified by an integer `id` and a `portable_id` (UUID).
Whiteboard	One revision of the design of a workflow: the graph, stored as XML in attribute `xml_export`.
Publish	Build the runnable workflow (steps, inputs, outputs, parameters, branches) from a whiteboard. Only a published revision runs.
Step	A node of the graph that executes an action.
Action plugin	The type of a step, for example `CustomRuby`. Also called step type or action type.
Action template	A configured instance of an action plugin: a database record holding the plugin settings (its attributes). Each step references one template by `Step_type` and `Action_id`.
Embedded template	A template that belongs to one workflow. Its name is `W<workflow id>__<step name>`.
Global template	A template shared by workflows, identified by its name.
Parameter	A named value of the workflow. A runtime parameter can be set when starting the workflow.
Input, Output	Named values consumed and produced by a step, as declared by the action plugin.
Work order	One execution of a published workflow.
Work step	One execution of a step in a work order.

Chapter 2

Quick start

The following file is a complete workflow with one step that returns a message. It uses the plain format, which is the simplest to write.

```
---
- !ruby/object:Workflow
  attributes:
    name: hello
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters></Parameters></Start>
      <Error x="900" y="350"
 ↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
 ↪  id="step1" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200"
 ↪  id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="400" y="50" id="step1"><Action_id>1</Action_id><Step_name>Say
 ↪  hello</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor><Prerequisites><Prerequ
 ↪  id="start"
 ↪  status="true"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters></Parameters><Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Say hello
      execute_code: "outputs['message'] = 'Hello'"
      typed_outputs: "{'message'=>'string'}"
- CustomRuby__1: []
- {}
- CustomRuby: 0.7.0
```

Import, publish and run it with the IBM Aspera CLI (`ascli`):

```
ascli orchestrator workflows import hello.yml          # displays the new workflow, including its id,
 ↪  for example 239
ascli orchestrator workflows publish 239
ascli orchestrator workflows start 239 @json:'{}' @json:'{"step":"Say hello","variable":"message"}'
```

The last command waits for the work order to finish and displays `Hello`. The same operations with the REST API are described in REST API.

Chapter 3

Lifecycle

1. Write a workflow file (`.yml`), or a package (`.wkf`) that also contains sub-workflows and remote nodes.
2. Import it: Orchestrator creates a new workflow, in draft state, with a new `id`. Each step's action template is created (or mapped to an existing global template).
3. Publish it: Orchestrator builds the runnable workflow from the last whiteboard revision.
4. Start it: Orchestrator creates a work order. Runtime parameters are given at this point.
5. Read the work order status and outputs.

Rules:

- Import and publish do not validate the graph. Publish only requires the whiteboard attribute `error_free` to be set. An invalid graph is published and fails at run time, or never ends. Use the authoring checklist.
- An import through `import_workflow` always creates a new workflow. If the name is already used, Orchestrator appends a space and `(import_1)`, `(import_2)`, and so on. The `portable_id` of the file is ignored and a new one is generated.
- To add a revision to an existing workflow instead, use `import_with_constraints` with `add as revision`. See Update an existing workflow.
- The API has no operation to delete a workflow: delete it in the user interface.
- Web UI & Draft vs Revision caveat: Importing via the API/CLI creates a published revision (`revision_id = 1`). When opening the workflow in the Web UI editor, the UI might initialize a new, empty *Draft* revision. Never click "Publish" on an empty draft in the Web UI, as this overwrites the workflow with an empty graph. To edit or view the imported graph in the Web UI, always open the imported revision explicitly under the Revisions tab and use *Rollback / Edit revision*, or manage publishing via `ascli`.

Chapter 4

Workflow file

4.1 File types

Extension	Content
`.yml`	One workflow, as described in this guide.
`.wkf`	A gzipped tar archive containing a manifest file `<name>.txt` and the files it lists. The manifest is a YAML list of file names: the first one is the main workflow (`.yml`), the others are its dependencies: sub-workflows (`Workflow_*.yml`) and remote nodes (`Remote_node*.node`). `export_workflow` produces this format when dependencies are exported.

The file name given to `import_workflow` must end with `yml` or `wkf`.

4.2 Structure

A workflow file is a YAML sequence of exactly six elements, in this order:

#	Element	Content
1	Workflow	Workflow attributes: name, description, ... See Workflow attributes.
2	Whiteboard	The graph (`xml_export`) and its state. See Whiteboard attributes.
3	Action templates	A map: key `<StepType>__<N>`, value the template object. See Action templates.
4	Dependency list	A map: same keys as element 3, value the list of external entities used by the template. Empty lists for a workflow without dependencies.
5	Packed dependencies	A map of dependencies included in a `.wkf` package. `{}` for a `.yml` file.
6	Plugin versions	A map: plugin name to version. Informational: not checked on import.

Rule for element 4 and 5: if element 4 lists dependencies and element 5 is empty, `import_workflow` does not import and returns the dependency list instead (see import_workflow). For a self-contained workflow, use empty lists in element 4.

The file can be written in two encodings. Both are accepted by the import.

4.3 Plain format

The plain format is the old (Orchestrator 4.0) format. It is the simplest to write, and the one used in this guide.

A file is in plain format when no line matches `concise_attributes:`. On import, Orchestrator converts it to the current format, line by line, and then loads it (`Compatibility::WorkflowYmlUpdater`). That converter is line-based, so the layout rules matter:

- An optional first line `---`.
- Each of the six elements starts with a dash and a space (`-` then space) at column 0.
  - Element 1 starts with `- !ruby/object:Workflow`.
  - Element 3 starts with `- <StepType>__<N>: !ruby/object:<StepType>` for the first template. Each next template starts with a line `<StepType>__<N>: !ruby/object:<StepType>` (any indentation). The separator must be exactly `: !ruby/object:`.
- A line `attributes:` is optional and ignored.
- The other lines of a section are its attributes, one `name: value` per line, with the same indentation within a section. Block scalars (`|`) are allowed, for example for `xml_export`.
- The class of a template is the part of its key before `__`.
- An unknown attribute name makes the import fail.
- In element 1, attributes with a null value are ignored.
- In element 3, attributes with a *blank* value are ignored: `null`, empty string, and also `false`. For a boolean attribute, write `1` or `0`, not `true` or `false`: `0` is kept, `false` is dropped (the database default then applies).
- Elements 4, 5 and 6 are required, even when empty.

On the server, the uploaded file is rewritten in the current format.

4.4 Rails format

This is the format produced by `export_workflow`: YAML serialization of Rails objects. Each attribute is an item of `concise_attributes`:

```
- !ruby/object:Workflow
  concise_attributes:
  - !ruby/object:ActiveModel::Attribute::FromDatabase
    name: name
    value_before_type_cast: My workflow
  - !ruby/object:ActiveModel::Attribute::FromDatabase
    name: label
  new_record: false
  active_record_yaml_version: 2
```

An attribute without `value_before_type_cast` is null. In element 3, each template has the same form, under its key (`- CustomRuby__2: !ruby/object:CustomRuby`).

To read an exported workflow, look for the attribute names given in this guide. To write a workflow, prefer the plain format.

4.5 Workflow attributes

Columns of a workflow, and how `import_workflow` uses them:

Attribute	Type	On import
`name`	string	Used. Suffixed with a space and `(import_N)` if already used.
`description`	text	Used.
`run_as`	string	Used. One of `SYSTEM`, `CREATOR`, `OPERATOR`.
`label`	string	Used.
`tags`	text	Used. Stored as `|tag1|tag2|`.

Attribute	Type	On import
templates	text	Used.
max_running	integer	Used. Maximum number of running work orders of this workflow.
cleanup_after_days	integer	Used.
purge_after_days	integer	Used.
user_lock, locked_by	boolean, integer	Used.
created_by	integer	Replaced by the importing user.
portable_id	string	Replaced by a new UUID.
source_revision	string	Replaced by <name>|<revision>.
comments	text	Replaced by Imported from file <path>.
id, folder_id, revision_id, generated_from, created_at, updated_at		Ignored. The folder is given by the import request.

The minimum is name, and run_as is recommended.

4.6 Whiteboard attributes

Attribute	Type	Rule
xml_export	text	Required. The graph. See Graph XML.
error_free	boolean	Required to publish. Publish returns false if it is null. Set it to true: it is not computed on import.
revision_id	integer	Optional. Defaults to 1.
created_by, workflow_id, revisions_log, id, created_at, updated_at		Ignored or replaced.

4.7 Action templates

Element 3 maps each key <StepType>__<N> to an action template.

- <StepType> is the plugin class name, for example CustomRuby, and must be the Step_type of the steps using it.
- <N> is a number unique within the file for that type, and must be the Action_id of the steps using it. On import, the template gets a new database id and the graph is updated: Action_id of steps, and action_id of Map_to and Carry_thru of type output.
- The value holds the template attributes: the plugin settings. The attribute names of each plugin are listed in the Action Template Reference.
- Attribute name decides how the template is imported:
  - W<any number>__<step name>: embedded template. It is created for the new workflow and renamed W<new workflow id>__<step name>. Use this form, with W0__ and the step name.
  - Any other name: global template. If a template of the same class and name exists and has the same attributes, it is reused. If it differs, a new template named <name> (import_N) is created. If none exists, it is created.
- Plugins that are not enabled on the server are enabled automatically by import_workflow. If that fails, the import returns the list of missing plugins.

How a template defines the inputs of its step is explained in Inputs of a step.

Chapter 5

Graph XML

The whiteboard attribute `xml_export` holds the graph as XML.

5.1 Skeleton

```
<?xml version="1.0" encoding="UTF-8"?>
<Workflow id="0">
<Start x="100" y="50" id="start"><Parameters></Parameters></Start>
<Error x="900" y="350"
 ↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
<End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites>...</Prerequisites></End>
<Fail x="900" y="200" id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
<Steps>
  <Step ...>...</Step>
</Steps>
<Parameters>...</Parameters>
<Notes></Notes>
</Workflow>
```

General rules:

- The root element name is free, `Workflow` by convention. Its attribute `id` is replaced on import: use `0`.
- The elements `Start`, `End`, `Fail` and `Error` are required, exactly once each.
- At least one `Step` is required.
- `Parameters` (top level) and `Notes` are optional.
- Attribute `id` of every node is a string, unique in the graph, used to link nodes. The designer uses UUIDs; any unique string works, for example `start` or `step1`.
- Attributes `x` and `y` are the position of the node in the designer. They do not change execution.
- Usual XML escaping applies (`&`, `<`, `"`). Values of parameters and of inputs mapped to a value are also URL-decoded: `%XX` and `%uXXXX` sequences are decoded. Write `%25` for a literal `%` followed by two hexadecimal digits.

5.2 Terminal nodes

Element	Required children	Meaning
`Start`	none (`Parameters` optional)	Entry point. Its `Parameters` links declare the global parameters (see Parameters).
`End`	`Synch_factor`, `Prerequisites`	Reached: the work order is `Complete`.
`Fail`	`Synch_factor`, `Prerequisites`	Reached: the work order is `Failed`.

Element	Required children	Meaning
`Error`	`Synch_factor`, `Prerequisites`	Reached: the work order is `Error`.

`Fail` and `Error` are also reached implicitly: when a work step ends with status `Failed` (or `Error`) and no step is connected to it for that status, the work order goes to `Fail` (or `Error`). So only success paths need to be connected to `End`; failures do not need explicit links.

5.3 Step

Each `Step` element has attributes `id`, `x`, `y`, and these children:

Child	Required	Content
`Action_id`	yes	Number `<N>` of the template key `<Step_type>__<N>`.
`Step_name`	yes	Name of the step, unique in the workflow. Used to name outputs (`<Step_name>:<output>`) and in `Map_to` of type `output`.
`Step_type`	yes	Plugin class name, for example `CustomRuby`.
`Synch_factor`	yes	Number of prerequisites to meet. See Prerequisites and branches. The designer adds attributes `x` and `y` (position of the join symbol).
`Prerequisites`	yes (to be valid)	Links from previous nodes.
`Inputs`	yes (may be empty)	Inputs of the step. See Inputs.
`Outputs`	yes (may be empty)	Output settings and carry-throughs. See Outputs.
`Parameters`	no	Links to step parameters. See Parameters.
`Version`	no	Plugin version, informational.
`Pre_processing`	no	Ruby code run before the action. See Processing code.
`Post_processing`	no	Ruby code run after the action.
`Timeout`	no	Execution timeout in seconds. Default: timeout of the plugin, else the system default.
`On_timeout`	no	Status on timeout: `Pass` (Complete), `Fail` (Failed, the default), `Error` (Error), `Raise` (Error, and the work order timeout action runs).
`Error_attempts`	no	Maximum attempts when the step ends with `Error` and a branch with status `Error` leaves it.
`Error_retryFor`, `Error_retryForUnit`	no	Retry period: value multiplied by unit (seconds).
`Error_retryDelay`	no	Delay between attempts, in seconds.
`Failed_attempts`, `Failed_retryFor`, `Failed_retryForUnit`, `Failed_retryDelay`	no	Same, for status `Failed`.
`comments`	no	Description of the step.
`weight`	no	Integer, used for progress display.

Child	Required	Content
`Journaling`	no	Journal entry settings (`Book`, `Event`, `Package`, `File`, `File_path`, `File_status`, `JournalStepMilestone`, `JournalProcessingTimeout`). A missing journal book is created on import.

Element names are case-sensitive and must be written exactly as above. The parser ignores unknown elements: a misspelled optional element has no effect.

5.4 Prerequisites and branches

A `Prerequisite` element links a previous node to the current one:

```
<Prerequisites><Prerequisite id="check" status="Complete"/></Prerequisites>
```

Attribute	Content
`id`	`id` of the previous node.
`status`	Status of the previous node that activates the link. `true` for a link from `Start`. Otherwise the final status of the previous step: `Complete`, `Failed` or `Error` (compared case-insensitively).
`router`	Line style in the designer. Use `draw2d.ManhattanConnectionRouter` for step prerequisites and `draw2d.BezierConnectionRouter` for parameter links. Ignored by execution engine, but required by the Web UI canvas to render connector lines.

`Synch_factor` sets how many prerequisites must be met:

Value	Meaning
`0`	All prerequisites (AND).
`N` > 0	At least `N` prerequisites. `1` is OR: for example `End` after alternative branches.
negative	Documented in the source as ”all but N”, but computed as ”number of prerequisites + N”: never met. Do not use.

A link from `Start` always counts as met.

A step can have several outgoing links (several steps referencing it): steps whose prerequisites are met run in parallel.

To branch on a condition, make a step end with `Complete` or `Failed`, and connect different steps to each status. The `Filter` plugin does exactly that with a Ruby expression; a `CustomRuby` step can set `@status`. See Recipe: branch on a status.

5.5 Synchronization rules (AND vs OR)

Choosing the correct `Synch_factor` on a join node depends on the execution pattern of incoming branches:

- Parallel nominal execution (`Synch_factor = 0` / AND): When two or more nominal branches execute simultaneously in parallel (e.g., a fan-out where several tasks run at the same time), set `Synch_factor` to `0` on the convergence step. The join node waits until all parallel branches have completed before proceeding.

- Mutually exclusive / Alternative branches (`Synch_factor = 1` / OR): When branches originate from mutually exclusive outcomes (e.g., an `if/else` condition, a route decision between alternative lanes, or a nominal branch vs an error/failure branch), only one branch runs per execution. Setting `0` deadlocks the workflow because it waits forever for the branches that were not executed. In this case, set `Synch_factor` to `1` on the join step (or `End`/`Fail`) so that it proceeds as soon as any one of the incoming branches completes.

5.6 Inputs

```
<Inputs>
  <Input name="person"><Map_to type="parameter" name="who"/></Input>
  <Input name="text"><Map_to type="output" name="greeting" step_type="CustomRuby" action_id="1"
↪  step_name="Greet"/><Processing>input_value = input_value + '!'</Processing></Input>
</Inputs>
```

Attribute `name` of `Input` is the name of an input of the step, as declared by the action template (see Inputs of a step). An `Input` whose name is not declared by the template is silently ignored.

`Map_to` gives the source of the value:

`type`	Other attributes	Source
`value`	`name`: the literal value	A constant. For an input of type `array`, a value not starting with `[` is wrapped into a one-element array.
`parameter`	`name`: parameter name	A parameter (see Parameters).
`global`	`name`: parameter name	Same as `parameter`.
`output`	`name`: output name, `step_name`, `step_type`, `action_id`	An output of a previous step. `step_name`, `step_type` and `action_id` must be the `Step_name`, `Step_type` and `Action_id` of that step.

Optional child `Processing`: Ruby code that transforms the value (see Processing code).

Every required input of the template must have an `Input` element.

5.7 Outputs

The outputs of a step are those declared by its action template, plus `Step_information`. They all exist, whether or not the `Outputs` element lists them. Each output is a variable named `<Step_name>:<output name>`.

The `Outputs` element only adds settings:

```
<Outputs>
  <Output name="file_name"><Processing>output_value = output_value.upcase</Processing></Output>
  <Carry_thru type="output" name="file" step_type="ArrayFanout" action_id="33" step_name="Fan out"/>
</Outputs>
```

Element	Content
`Output`	Attribute `name`. Optional children: `Processing` (Ruby code transforming the value), `Global` (name of a global variable that also receives the value).
`Carry_thru`	Makes a variable available as an input of this step without the plugin declaring it. Attributes: `type` (`output` or `parameter`), `name`, and for `output`: `step_name`, `step_type`, `action_id`. Used for example after an `ArrayFanout`, to pass the current element through the steps of the fan-out.

Output `Step_information` is a `hash` with keys `step_id`, `step_name`, `step_status`, `step_status_details`, `attempts`, `processingTime`, `elapsedTime`, `completionTime`.

5.8 Parameters

A parameter is defined once in the top-level `Parameters` element, and linked to `Start` (global parameter) or to a step (step parameter):

```
<Start x="100" y="50" id="start"><Parameters><Parameter id="p_who"/></Parameters></Start>
...
<Parameters>
  <Parameter x="100" y="200" id="p_who">
    <Name>who</Name><Value>world</Value><Value_type>string</Value_type>
    <Runtime_override>true</Runtime_override><Runtime_optional>true</Runtime_optional>
  </Parameter>
</Parameters>
```

Definition (`Parameter` with attributes `id`, `x`, `y`):

Child	Content
`Name`	Name, used in `Map_to` of type `parameter` and when starting the workflow.
`Value`	Value at design time, or default value of a runtime parameter. Use JSON for `array` and `hash`, for example `["/tmp/a", "/tmp/b"]`.
`Value_type`	One of the value types.
`Runtime_override`	`true`: the value can be given when starting the workflow. Default `false`.
`Runtime_optional`	`true`: a runtime value is optional, and `Value` is used when absent. Default `false`.
`Comments`	Description.

Link (`Parameter` with attribute `id`, and `router="draw2d.BezierConnectionRouter"` for the designer) in `Start/Parameters` or `Step/Parameters`.

Rules:

- A parameter that is not linked to `Start` or to a step does not exist after publish: inputs mapped to it are silently not created.
- Runtime parameters are the inputs of the workflow: `workflow_inputs_spec` lists them, and `initiate` receives them as `external_parameters`.

5.9 Value types

Type	Content
`string`	Text.
`int`	Integer.
`float`	Floating point number.
`flag`	Boolean.
`date`	Date and time.
`array`	List (JSON array).
`hash`	Map (JSON object).
`object`	Any serializable value.
`pwd`	Password: hidden in the user interface, masked or encrypted on export.
`attachment`	File attachment.

5.10 Notes

`Notes` holds designer notes: `<Note id="..." note_id="..." x="..." y="..." collapsed="..."><Text>...</Text></Note>`. They do not change execution.

Chapter 6

Inputs of a step

The inputs of a step are declared by its action template, in method `inputs_spec` of the plugin: a map of required inputs and a map of optional inputs, each from name to value type. The Action Template Reference lists them for each plugin.

Many plugins derive inputs from some of their template attributes (`ActionTools#default_inputs_spec`):

- A text attribute left blank in the template becomes an input, so its value can come from the graph at run time. The Action Template Reference gives its name in column Input if blank.
  - Input name: the value of the plugin constant `VAR_<ATTRIBUTE>` if it exists, else the attribute name with its first letter in upper case (for example `file_path` gives `File_path`).
  - The input is optional if the plugin defines a constant `DEFAULT_<ATTRIBUTE>`, else required.
  - An attribute name ending with `_<type>` (for example `delay_int`) gives an input of that type, named without the suffix.
- A text attribute that is set can contain placeholders `<%= variable %>`: each one creates an input.
- A non-text attribute left null becomes an input of the corresponding type.

So the same plugin can have different inputs depending on its template. For example, `LocalFileWatcher` with `directory` left blank has a required input `directory`.

Plugins that let the template declare its inputs and outputs explicitly (for example `CustomRuby` and `Filter`) take them as a type map in text: JSON (`{"person":"string"}`) or Ruby hash syntax (`{'person'=>'string'}`).

Before calling the action, Orchestrator adds these inputs: `OUTPUTS` (names and types of the outputs), `WORKORDER_ID` (the work step id), `ATTEMPT`, `RUNNING_AS`.

6.1 Placeholders

A text attribute marked `placeholders: true` in plugins.yml can contain ERB placeholders. Each variable they name becomes an input of the step, and the plugin replaces the placeholders with the input values at run time. For example, an `EmailNotification` template with `template` (the message body) set to `Hello <%= user_name %>, <%= file_name %> is ready.` gives required inputs `user_name` and `file_name`.

Placeholders in an attribute that is not marked create no input. This is the case of some attributes that give an input when left blank.

These forms create inputs:

Placeholder	Creates input	Replaced by
`<%= name %>`	`name`	The value of input `name`.

Placeholder	Creates input	Replaced by
`<%= function(a, b) %>`	`a`, `b`	The result of `function` called with the values of inputs `a` and `b`. Arguments are input names, not literal values. `function` is a method of the plugin (for example `proxy_tag` of `ItunesTransporter`), `xml` (a hash as XML elements: `{'v'=>[1, 2]}` gives `<v>1</v><v>2</v>`), or a method with its class (for example `File.basename`).
`var(name)` in an expression	`name`	`var(name)` returns the value of input `name`, for example `<%= var(file_count).to_i + 1 %>`.

Any other Ruby code in `<%= ... %>` runs, but creates no input: in `<%= file_name.upcase %>`, `file_name` is not an input of the step, unless another placeholder declares it. Write `<%= var(file_name).upcase %>`. `var` is only for expressions: `<%= var(file_name) %>` alone fails, write `<%= file_name %>`.

The input created by a placeholder:

- Has the name written in the placeholder: letters, digits and `_`.
- Is a `string`, unless the name ends with `_<type>`: `<%= count_int %>` gives input `count_int` (the suffix is kept) of type `int`.
- Is required, unless the plugin defines a constant `DEFAULT_<NAME>`.
- Is not created if the name is a method of the template that returns a value: `name` (the template name) or another attribute of the template that is set. That value is used.

Some plugins scan placeholders with their own code instead of `default_inputs_spec` (for example `Filter`): the input is a required `string`, unless the template declares it otherwise (`mandatory_inputs` and `optional_inputs` of `Filter`).

Placeholders are scanned only in the value set in the template: the ones in `code_default` (attribute left blank) create no input.

At run time:

- An input without a value is replaced by an empty string.
- An array is written in Ruby notation: `[1, 2]`.
- `<% ... %>` tags (code without output, for example `<% if ... %>`) run only when the value also contains an expression such as `<%= var(file_name).upcase %>`: do not rely on them.

6.2 Plugin specifications

The file plugins.yml gives, for each plugin, its template attributes and the inputs and outputs of its steps, with the template settings they depend on. They were found by calling methods `inputs_spec` and `outputs_spec` of the plugin on templates with one or two attributes changed. Read one plugin with `ascli orchestrator plugins spec <PluginClass>`, or with `yq '.<PluginClass>' plugins.yml`.

Each plugin is a key, its class name: the `Step_type` of a step, and the class of its template (key `<PluginClass>__<N>`).

```
AmazonS3Operation:
  attributes:
    operation: {type: string, label: File Operation, widget: select, values: [upload, delete, list,
↪   ...]}
    aws_key: {type: string, hint: Enter AWS access key ID}
  inputs:
```

```
    Aws_key: {type: string, required: true, if_blank: aws_key}
    Local_file_path: {type: string, required: true, if_blank: local_file_path, when: {operation:
↪   {in: [upload, download]}}}
  outputs:
    ETag: {type: string, when: {operation: {in: [upload, copy object]}}}
```

Attributes (besides `name` and `comments`, common to all templates):

Field	Content
`type`	Database column type. In plain format, write a boolean as `1` or `0`.
`label`	Label in the template editor, when it is not the attribute name (`aws_key`: `Aws key`).
`widget`	When not text or boolean: `select`, `multiline_text`, `password`, `number`, `remote_node` (name of a remote node), `type_map` (`{'name'=>'type'}`), `list`, `hash`, `file`, `action_template` (name of a template).
`values`	Allowed values.
`dynamic_values`	`true`: allowed values come from the server or an external system.
`run_time_value`	For a `remote_node`: value that makes the node an input of the step.
`default`	Database default.
`code_default`	Value used by the plugin when the attribute is blank.
`hint`	Placeholder text in the template editor.
`format`	Language of the value (code editor).
`placeholders`	`true`: placeholders in the value create inputs (`<%= variable %>`, `<%= function(variable, ...) %>`, `var(variable)` in an expression). Not set: placeholders in the value are not scanned, even if the attribute gives an input when blank.
`declares`	The value declares inputs or outputs of the step (`required_inputs`, `optional_inputs`, `outputs`), as a type map or a list.

Inputs and outputs (a name in angle brackets is the value of that template attribute):

Field	Content
`type`	Value type.
`required`	Inputs: `true`, `false`, or the condition for which the input is required (`{if_blank: ...}`, `{if_set: ...}`, `{when: ...}`).
`if_blank`	Exists only when this attribute is blank (for an input: it replaces the attribute), or when all attributes of the list are blank.
`if_set`	Exists only when this attribute (or all of the list) is not blank.
`when`	Exists only for some values of attributes: `{<attribute>: {in: [...]}}` (`null`: attribute blank) or `{<attribute>: {not_in: [...]}}` (set to another value).
`conditional`	`true`: other template settings also decide whether it exists, the conditions given are not complete.

`if_blank`, `if_set` and `when` must all match, unless `conditional` is `true`.

Chapter 7

Execution

7.1 Statuses

A work step ends with one of the following statuses, which are matched by the `status` of prerequisites:

Status	Meaning
`Complete`	Success.
`Failed`	Functional failure (for example a filter evaluated to false).
`Error`	Technical error (exception, timeout with `On_timeout` set to `Error`).

While running, a work step can also be `Inactive`, `In Progress`, `Paused`, `Canceling`, and so on. A work order is `Complete`, `Failed` or `Error` when it reaches `End`, `Fail` or `Error`, and `Canceled` when canceled.

7.2 Retries

When a step ends with `Error` (or `Failed`) and a branch with that status leaves it, the step can be retried before that branch is followed: up to `Error_attempts` (or `Failed_attempts`) attempts, within `Error_retryFor` x `Error_retryForUnit` seconds, waiting `Error_retryDelay` seconds between attempts.

7.3 Processing code

Ruby code can be attached to a step, an input or an output. It runs inside Orchestrator: it has full access to the server.

Location	Runs	Variables
`Input/Processing`	When the input value is read.	`input_name` (read-only), `input_value` (set it to change the value).
`Step/Pre_processing`	Before the action.	`@inputs`: map of all input values, modifiable. `@work_order_id`, `@state_id` (work step id).
`Step/Post_processing`	After the action.	`outputs`: map of output values, modifiable. `@status`, `@status_details`: set them to change the step result. `@work_order_id`, `@state_id`.
`Output/Processing`	When the output value is stored.	`output_name` (read-only), `output_value` (set it to change the value).

If processing code raises an exception, a warning is logged and execution continues with the unmodified value, unless the server option `strict_prepost_processing` is set, in which case the step ends with `Error`.

7.4 Triggers

Some plugins wait for an event instead of executing at once, for example `LocalFileWatcher`. They do not hold a worker while waiting. If the template enables continuous monitoring (attribute `allow_multiple` of `LocalFileWatcher`), each event starts a new work order that continues with the next steps, while the trigger keeps waiting.

Chapter 8

Main action plugins

The Action Template Reference lists the attributes, inputs and outputs of all plugins. This section details the plugins most useful to write workflows.

8.1 CustomRuby

Runs Ruby code. Inputs and outputs are declared in the template.

Attribute	Content
`execute_code`	Ruby code of the action.
`mandatory_inputs`	Required inputs: type map, for example `{'person'=>'string'}`.
`optional_inputs`	Optional inputs: type map.
`typed_outputs`	Outputs: type map, for example `{'greeting'=>'string'}`.
`inputs_spec_code`	Instead of `mandatory_inputs` and `optional_inputs`: Ruby code returning a map (required inputs) or an array of two maps (required, optional).
`outputs_spec_code`	Instead of `typed_outputs`: Ruby code returning the output type map.
`validate_inputs_code`	Ruby code returning `true` if the inputs are valid.
`use_code_from_github`, `github_*`, `source_control_type`, `use_cached`, `verify_ssl`	Get the code from a GitHub or Bitbucket repository instead of `execute_code`.

Contract of `execute_code`:

- Input values are in the map `@inputs`, for example `@inputs['person']`. An input can also be read as a method with its name, unless that name is also a method of the template: `name` returns the template name, not an input named `name`. Prefer `@inputs[...]`.
- Set outputs in the map `outputs` (or `@outputs`): `outputs['greeting'] = 'Hello'`. Only outputs declared in `typed_outputs` are stored.
- Optionally set `@status` to `Complete` (default), `Failed` or `Error`, and `@status_details` to a message. Any other status becomes `Error`.
- An exception ends the step with `Error`.

8.2 Filter

Evaluates a Ruby expression. The step ends with `Complete` if the result is true, `Failed` otherwise, and output `FilterResult` (`flag`) holds the result.

Attribute	Content
command	Ruby expression. Placeholders create required string inputs, unless declared in mandatory_inputs or optional_inputs. They are replaced by text before the expression runs: quote a string value, as in '<%= Status %>' == 'ok'.
mandatory_inputs, optional_inputs	Additional inputs: type maps.
filter_outputs	Additional outputs: type map.

8.3 LocalFileWatcher

Waits for files matching a pattern in a directory of the Orchestrator host. Output FileName is the full path of the file (string), or the list of paths (array) if return_all is set; FileContent holds the file content if read_file is set.

Attribute	Content
directory	Directory to watch. Blank: becomes input directory.
file_name	File name pattern (glob, or regular expression if use_regex_matching). Blank: becomes input File_name.
allow_multiple	Continuous monitoring: start a new work order for each detection, and keep watching.
remove_file	Delete the file once detected.
archive_directory, archive_with_timestamp, keep_timestamps	Move the file to an archive directory once detected.
cool_off	Seconds a file must stay unchanged to be considered stable.
polling_frequency	Seconds between scans (default 5).
partial_file	Trigger on files still being written.
return_all	Return all matching files at once.
trigger_type	Persistence of past triggers: by workorder group (default), by workflow, by action template, by step, none.
exclusions, fullpath_exclusions, ignore_zero_bytes, ignore_readlocked	Filters.

Pitfall: with remove_file (and no archive directory) not set, a detected file stays in the directory. Past triggers are remembered per work order group by default, and each new work order is a new group: each new work order detects the same file again at once. Set remove_file, or archive_directory, so that a file is processed once.

8.4 ArrayFanout and Funnelin

ArrayFanout runs the next steps once per element of an array; Funnelin waits for all those executions and gathers their results.

- ArrayFanout: attribute fanout_variable is the name of the input (array), fanned_as the name of the output holding the current element, variable_type its type, serialize to run sequentially. Output Fanout_Size is the number of elements.
- Funnelin: attribute funnel_variable (and funnel_type) is the name of the input collected from each execution, funneled_inputs additional inputs (type map), return_status to add status outputs, fail_if_failures to fail if an execution failed. Output Hash_by_state_id maps each work step id to its value; each additional input x gives an output x_hash.

The steps between them use Carry_thru to pass the current element along.

Chapter 9

Recipes

The following workflows were imported, published and run on Orchestrator 4.1.6.

9.1 Recipe: minimal workflow

See Quick start.

9.2 Recipe: runtime parameter and step chaining

Step `Greet` receives the runtime parameter `who`; step `Shout` receives the output of `Greet`, with input processing.

```
---
- !ruby/object:Workflow
  attributes:
    name: greet
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_who"/></Parameters></Start>
      <Error x="900" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="shout" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200"
↪   id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50"
↪   id="greet"><Action_id>1</Action_id><Step_name>Greet</Step_name><Step_type>CustomRuby</Step_type><Synch_f
        <Prerequisites><Prerequisite id="start" status="true"/></Prerequisites>
        <Parameters></Parameters>
        <Inputs><Input name="person"><Map_to type="parameter" name="who"/></Input></Inputs>
        <Outputs></Outputs></Step>
      <Step x="600" y="50"
↪   id="shout"><Action_id>2</Action_id><Step_name>Shout</Step_name><Step_type>CustomRuby</Step_type><Synch_f
        <Prerequisites><Prerequisite id="greet" status="Complete"/></Prerequisites>
        <Parameters></Parameters>
        <Inputs><Input name="text"><Map_to type="output" name="greeting" step_type="CustomRuby"
↪   action_id="1" step_name="Greet"/><Processing>input_value = input_value +
↪   '!'</Processing></Input></Inputs>
        <Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
↪   id="p_who"><Name>who</Name><Value>world</Value><Value_type>string</Value_type><Runtime_override>true</Ru
```

```
      <Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Greet
      mandatory_inputs: "{'person'=>'string'}"
      execute_code: "outputs['greeting'] = \"Hello #{@inputs['person']}\""
      typed_outputs: "{'greeting'=>'string'}"
  CustomRuby__2: !ruby/object:CustomRuby
    attributes:
      name: W0__Shout
      mandatory_inputs: "{'text'=>'string'}"
      execute_code: "outputs['result'] = @inputs['text'].upcase"
      typed_outputs: "{'result'=>'string'}"
- CustomRuby__1: []
  CustomRuby__2: []
- {}
- CustomRuby: 0.7.0
```

```
ascli orchestrator workflows start <id> @json:'{"who":"Laurent"}'
 ↪  @json:'{"step":"Shout","variable":"result"}'   # HELLO LAURENT!
ascli orchestrator workflows start <id> @json:'{}' @json:'{"step":"Shout","variable":"result"}'
 ↪  # HELLO WORLD!
```

9.3 Recipe: branch on a status

Step `Check` ends with `Failed` when the runtime parameter `value` is empty; one step is connected to each status; `End` waits for any of them (`Synch_factor` 1).

```
---
- !ruby/object:Workflow
  attributes:
    name: branch
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters><Parameter id="p_value"/></Parameters></Start>
      <Error x="1000" y="350"
 ↪  id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="1000" y="50" id="end"><Synch_factor>1</Synch_factor><Prerequisites><Prerequisite
 ↪  id="on_ok" status="Complete"/><Prerequisite id="on_failed"
 ↪  status="Complete"/></Prerequisites></End>
      <Fail x="1000" y="200"
 ↪  id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="300" y="50"
 ↪  id="check"><Action_id>1</Action_id><Step_name>Check</Step_name><Step_type>CustomRuby</Step_type><Synch_f
        <Prerequisites><Prerequisite id="start"
 ↪  status="true"/></Prerequisites><Parameters></Parameters>
        <Inputs><Input name="value"><Map_to type="parameter"
 ↪  name="value"/></Input></Inputs><Outputs></Outputs></Step>
      <Step x="600" y="0" id="on_ok"><Action_id>2</Action_id><Step_name>On
 ↪  OK</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="check"
 ↪  status="Complete"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      <Step x="600" y="150" id="on_failed"><Action_id>3</Action_id><Step_name>On
 ↪  Failed</Step_name><Step_type>CustomRuby</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="check"
 ↪  status="Failed"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters><Parameter x="100" y="200"
 ↪  id="p_value"><Name>value</Name><Value></Value><Value_type>string</Value_type><Runtime_override>true</Run
```

```
      <Notes></Notes>
      </Workflow>
- CustomRuby__1: !ruby/object:CustomRuby
    attributes:
      name: W0__Check
      optional_inputs: "{'value'=>'string'}"
      execute_code: "@status = @inputs['value'].to_s.empty? ? 'Failed' : 'Complete'"
  CustomRuby__2: !ruby/object:CustomRuby
    attributes:
      name: W0__On OK
      execute_code: "outputs['result'] = 'value was provided'"
      typed_outputs: "{'result'=>'string'}"
  CustomRuby__3: !ruby/object:CustomRuby
    attributes:
      name: W0__On Failed
      execute_code: "outputs['result'] = 'value was empty'"
      typed_outputs: "{'result'=>'string'}"
- CustomRuby__1: []
  CustomRuby__2: []
  CustomRuby__3: []
- {}
- CustomRuby: 0.7.0
```

With `{"value":"x"}`, only `On OK` runs; with `{}`, only `On Failed` runs. Both work orders end `Complete`.

9.4 Recipe: wait for a file

The workflow waits for a file matching `/tmp/ascli_watch_*.txt` on the Orchestrator host, deletes it, and returns its path.

```
---
- !ruby/object:Workflow
  attributes:
    name: wait_for_file
    run_as: SYSTEM
- !ruby/object:Whiteboard
  attributes:
    error_free: true
    xml_export: |
      <?xml version="1.0" encoding="UTF-8"?>
      <Workflow id="0">
      <Start x="100" y="50" id="start"><Parameters></Parameters></Start>
      <Error x="900" y="350"
↪   id="error"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Error>
      <End x="900" y="50" id="end"><Synch_factor>0</Synch_factor><Prerequisites><Prerequisite
↪   id="wait" status="Complete"/></Prerequisites></End>
      <Fail x="900" y="200"
↪   id="fail"><Synch_factor>0</Synch_factor><Prerequisites></Prerequisites></Fail>
      <Steps>
      <Step x="400" y="50" id="wait"><Action_id>1</Action_id><Step_name>Wait for
↪   file</Step_name><Step_type>LocalFileWatcher</Step_type><Synch_factor>0</Synch_factor>
        <Prerequisites><Prerequisite id="start"
↪   status="true"/></Prerequisites><Parameters></Parameters><Inputs></Inputs><Outputs></Outputs></Step>
      </Steps>
      <Parameters></Parameters><Notes></Notes>
      </Workflow>
- LocalFileWatcher__1: !ruby/object:LocalFileWatcher
    attributes:
      name: W0__Wait for file
      directory: /tmp/
      file_name: ascli_watch_*.txt
      allow_multiple: 0
      remove_file: 1
      cool_off: 2
- LocalFileWatcher__1: []
```

```
- {}
- LocalFileWatcher: 1.9.3
```

```
ascli orchestrator workflows start <id> @json:'{}' @json:'{"step":"Wait for
↪  file","variable":"FileName"}'
```

The command returns when a matching file has been stable for `cool_off` seconds, for example `/tmp/ascli_watch_20261006_230726.txt`.

9.5 Update an existing workflow

`import_workflow` always creates a new workflow. To add a revision to an existing workflow, the file must be on the Orchestrator host, and `import_with_constraints` is called with `add as revision`:

1. Have the file on the server. `import_workflow` stores each uploaded file in `<run_dir>/workflows/import/<import_file_name>` (for example `/opt/aspera/orchestrator/var/run/orchestrator/workflows/import/`), but also imports it as a new workflow.
2. Call `import_with_constraints` with the path of the file and the id of the workflow to update (see import_with_constraints). The response is the workflow; its last revision is the imported one.
3. Publish the workflow: the new revision becomes the running one.

With `ascli`:

```
ascli orchestrator workflows import_with_constraints
↪  @json:'{"filename":"/opt/aspera/orchestrator/var/run/orchestrator/workflows/import/my_workflow.
↪  yml","add_as_revision":239}'
ascli orchestrator workflows publish 239
```

Chapter 10

Authoring checklist

Import and publish do not check these rules. The designer checks most of them (`Whiteboard#parse_workflow`) before setting `error_free`.

File:

- Six elements, in order; elements 4, 5 and 6 present, even empty.
- Plain format: no line matching `concise_attributes:`; booleans of templates written `1` and `0`.
- `error_free: true` in the whiteboard.
- Each template key `<StepType>__<N>` matches the `Step_type` and `Action_id` of its steps.
- Embedded templates named `W0__<step name>`.

Graph:

- `Start`, `End`, `Fail`, `Error` present once; at least one `Step`.
- `End`, `Fail`, `Error` and each `Step` have `Synch_factor`; each `Step` has `Inputs` and `Outputs` (may be empty).
- Each step has a name, unique in the workflow, and a type of an enabled plugin.
- Each step has at least one prerequisite (else it never runs: ”Step is orphaned”).
- Each step is the prerequisite of at least one node with status `Complete` or `true` (else: ”Step does not have output”).
- At least one prerequisite reaches `End`, `Fail` or `Error` (else: ”Workflow does not terminate”).
- Each `Map_to` of type `output` refers to an existing step, by `step_name`, `step_type` and `action_id`.
- Each `Map_to` of type `parameter` refers to a defined parameter, linked to `Start` or a step.
- Each required input of each template is mapped.
- Each parameter has a `Name` and a `Value_type`; a parameter that is not a runtime parameter, or that is an optional runtime parameter, has a `Value`.
- Join nodes after mutually exclusive / alternative branches have `Synch_factor` 1; join nodes after concurrent parallel nominal branches have `Synch_factor` 0.
- For optimal Web UI rendering, use `router="draw2d.ManhattanConnectionRouter"` on step prerequisites and `router="draw2d.BezierConnectionRouter"` on parameter links.
- Retry settings (`Error_*`, `Failed_*`) only on steps that have an outgoing branch with that status.

Chapter 11

REST API

Base URL: `https://<host>/aspera/orchestrator/api/`. Authentication: a JWT obtained with `POST api/login`, or query parameters `login` and `password` when JWT authentication is disabled. The response format is chosen with query parameter `format` (`xml`, the default, or `json`).

Routes: each endpoint below has a route with the given method. In addition, any endpoint can be called as `api/<endpoint>/<id>`, with any HTTP method, where `<id>` is the main identifier (workflow or work order).

Errors are returned with the exception message and stack trace in the response body.

11.1 import_workflow

`POST api/import_workflow`, body `multipart/form-data`:

Field	Required	Content
`import_file`	yes	The workflow file (file part).
`import_file_name`	yes	File name, ending with `yml` or `wkf`. The upload is stored under this name in `<run_dir>/workflows/import/`.
`folder_id`	no	Folder of the new workflow.
`link_to_subworkflows`	no	Link to existing sub-workflows instead of importing them.
`publish_workflow`	no	`true` or `1`: publish after import. The server option `publish_api_imported_workflow` sets the default.

Response: the new workflow as JSON (`{"workflow": {"id": ..., "name": ..., ...}}`), in both formats. Other responses with status 200:

- `{"plugins": [...], "missing_deps": [...]}`: plugins that could not be enabled; nothing is imported.
- `{"dependencies": ..., "file": ..., "workflow": ...}`: the file lists dependencies that are not packed; nothing is imported.

On error: status 500 with `{"error": ..., "message": ..., "details": ...}`.

11.2 export_workflow

`GET api/export_workflow/<id or portable id>` (no route without the id in the path)

Returns the workflow file (`.yml`, Rails format). With parameter `export_with_dependencies`, returns a `.wkf` package with sub-workflows and remote nodes. Any non-empty value of `export_with_dependencies`, including `false`, enables it.

11.3 publish_workflow

`POST api/publish_workflow?id=<id or portable id>`, or `api/publish_workflow/<id or portable id>` with any method. The id can also be given in a JSON body `{"id": ...}`.

Publishes the last whiteboard revision. Response: `true` or `false` (JSON), or `<WorkflowPublish><Success>true</Success></WorkflowPublish>` (XML). `false` means the whiteboard has no `error_free` value.

11.4 find_constraints

`GET api/find_constraints?file=<path>[&wf_id=<workflow id>]`

Analyzes a workflow file present on the Orchestrator host and returns the list expected by `import_with_constraints`, with the possible choices for each conflict:

```
[{"filename": "<path>"}, {"add as revision": null}, {"subwf constraints": {}},
 {"action template constraints": {}}, {"remote node constraints": {}},
 {"missing plugins": null, "Auto-enable missing plugins?": null}]
```

It does not check that the file exists: a wrong path returns the same empty lists.

11.5 import_with_constraints

`POST api/import_with_constraints`, JSON body: an array of six objects, in this order:

```
[{"filename": "/opt/aspera/orchestrator/var/run/orchestrator/workflows/import/my_workflow.yml"},
 {"add as revision": 239},
 {"subwf constraints": {}},
 {"action template constraints": {"Search Files": "Map to the existing Global Template"}},
 {"remote node constraints": {"Baton": "Map to the existing Remote Node"}},
 {"Auto-enable missing plugins?": "true"}]
```

Item	Content
`filename`	Path of the file on the Orchestrator host.
`add as revision`	Id of an existing workflow to add a revision to, or `null` to create a new workflow.
`subwf constraints`	Sub-workflow name to choice: `Map to the Existing Sub-Workflow`, `Create a new Sub-Workflow with a unique name`, `Add a new revision to the existing Sub-Workflow`.
`action template constraints`	Global template name to choice: `Map to the existing Global Template`, `Create a new Global Template with a different name`, `Override the existing Global Template with this new template`.
`remote node constraints`	Remote node name to choice: `Map to the existing Remote Node`, `Create a new Remote Node with a different name`, `Override the Existing Remote Node with this importing node`.
`Auto-enable missing plugins?`	`true` or `false`.

Response: the workflow as JSON. The workflow is not published.

11.6 initiate

`POST api/initiate?format=json`, JSON body `{"workflow_id": <id>, "external_parameters": {"<name>": <value>, ...}}`. Query parameters:

Parameter	Content
`synchronous`	`true`: wait for the end of the work order.
`timeout`	With `synchronous`: maximum wait, in seconds.
`explicit_output_step`, `explicit_output_variable`	With `synchronous`: return only this output.
`explicit_output`	Same, as `<step name>:<output name>`.
`on_error_return_code`, `on_failed_return_code`	HTTP status to return when the work order ends `Error` or `Failed`.
`tags`	Comma-separated tags of the work order.

11.7 Other workflow endpoints

Endpoint	Content
`GET api/workflows_list`	All workflows: id, name, portable id, published and latest revision.
`GET api/workflow_inputs_spec/<id>`	Runtime parameters: name, type, optional, default value.
`GET api/workflow_outputs_spec/<id>`	Outputs: `<step name>:<output name>`, type.
`GET api/work_orders_list/<workflow id>`	Work orders of a workflow.
`GET api/work_order_status/<id>`	Status of a work order.
`GET api/work_order_output/<id>`	Output values of a work order (XML: one `variable` element per output).
`GET api/work_order_cancel/<id>`	Cancel a work order.

11.8 ascli commands

Operation	Command
Import	`ascli orchestrator workflows import <file>`
Import with constraints, or as a revision	`ascli orchestrator workflows import_with_constraints @json:'{"filename":"<path on server>","add_as_revision":<id>}'`
Export	`ascli orchestrator workflows export <id> --out.file=<file>`
Export with dependencies (`.wkf`)	`ascli orchestrator workflows export <id> @json:'{"dependencies":true}' --to-folder=<folder>`
Publish	`ascli orchestrator workflows publish <id>`
Inputs, outputs	`ascli orchestrator workflows inputs <id>`, `ascli orchestrator workflows outputs <id>`
Start	`ascli orchestrator workflows start <id> [<parameters>] [<execution>]`, where execution is `{"synchronous":true}` or `{"step":"<step name>","variable":"<output name>"}`
Work order	`ascli orchestrator workorders status <id>`, `ascli orchestrator workorders output <id>`, `ascli orchestrator workorders cancel <id>`

Chapter 12

Known issues

Observed in Orchestrator 4.1.6:

- A negative `Synch_factor` is never met (see Prerequisites and branches).
- `find_constraints` returns empty constraints for a file that does not exist.
- `export_workflow` exports dependencies for any non-empty value of `export_with_dependencies`, including `false`.
- `export_workflow` has no route without the workflow id in the path, although its source comment documents `?id=`.
- The `.wkf` package produced by `export_workflow` with dependencies contains an empty copy of itself (the archive is built in the folder it packs).
- API errors return the exception stack trace, and often status 404 or 500 regardless of the cause.
- The example files of the product (`docs/workflow_examples`) contain `<Failed_retry_for_unit>`, while the parser reads `<Failed_retryForUnit>`: that setting is ignored.

End of document